

Лабораторная №1

Задание 1:



1. MANAGEMENT (Управление):

а. Это раздел для общего администрирования сервера.
Здесь вы можете:

- **Server Status:** Смотреть общую информацию о состоянии сервера.
- **Users and Privileges:** Создавать, удалять и настраивать права пользователей.
- **Data Export / Import:** Создавать резервные копии баз данных и восстанавливать их.

2. INSTANCE (Экземпляр БД):

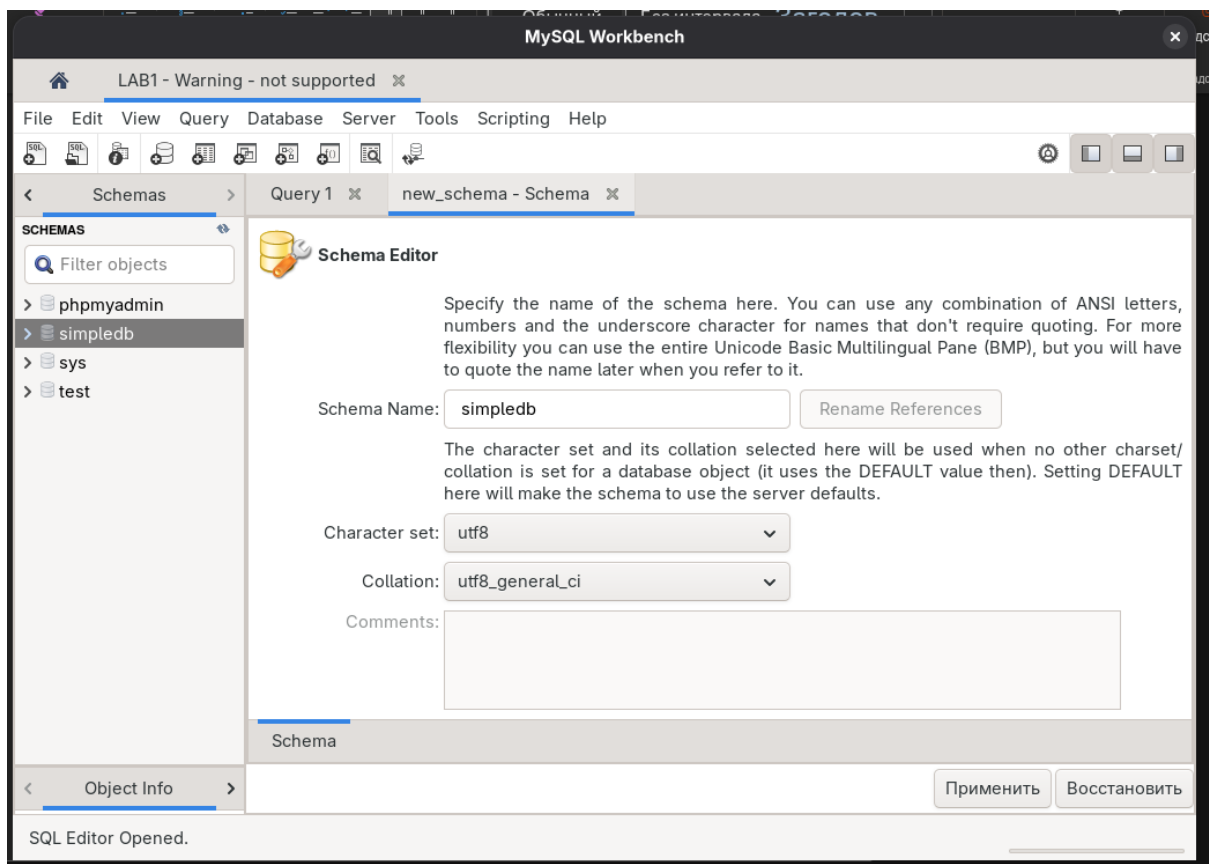
- Этот раздел отвечает за управление самим процессом сервера. Отсюда можно:
 - **Startup / Shutdown:** Запускать и останавливать сервер баз данных.

- **Server Logs:** Просматривать файлы логов, чтобы найти ошибки или информацию о работе сервера.
- **Options File:** Редактировать конфигурационный файл сервера.

3. PERFORMANCE (Производительность):

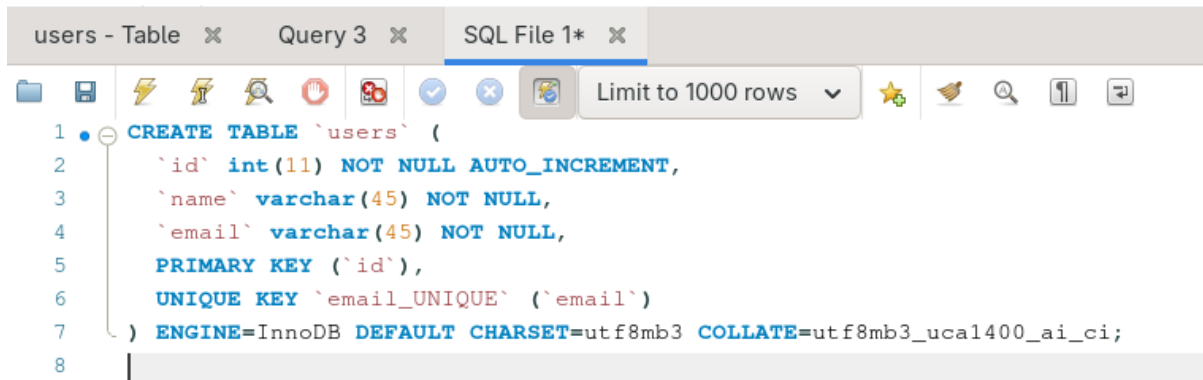
- Раздел для мониторинга производительности сервера. Позволяет:
 - **Dashboard:** Видеть графики и показатели производительности в реальном времени (нагрузка на процессор, использование памяти и т.д.).
 - **Performance Reports:** Получать подробные отчеты, чтобы найти медленные запросы и другие проблемы с производительностью.

Задание 2: создана база данных simpledb



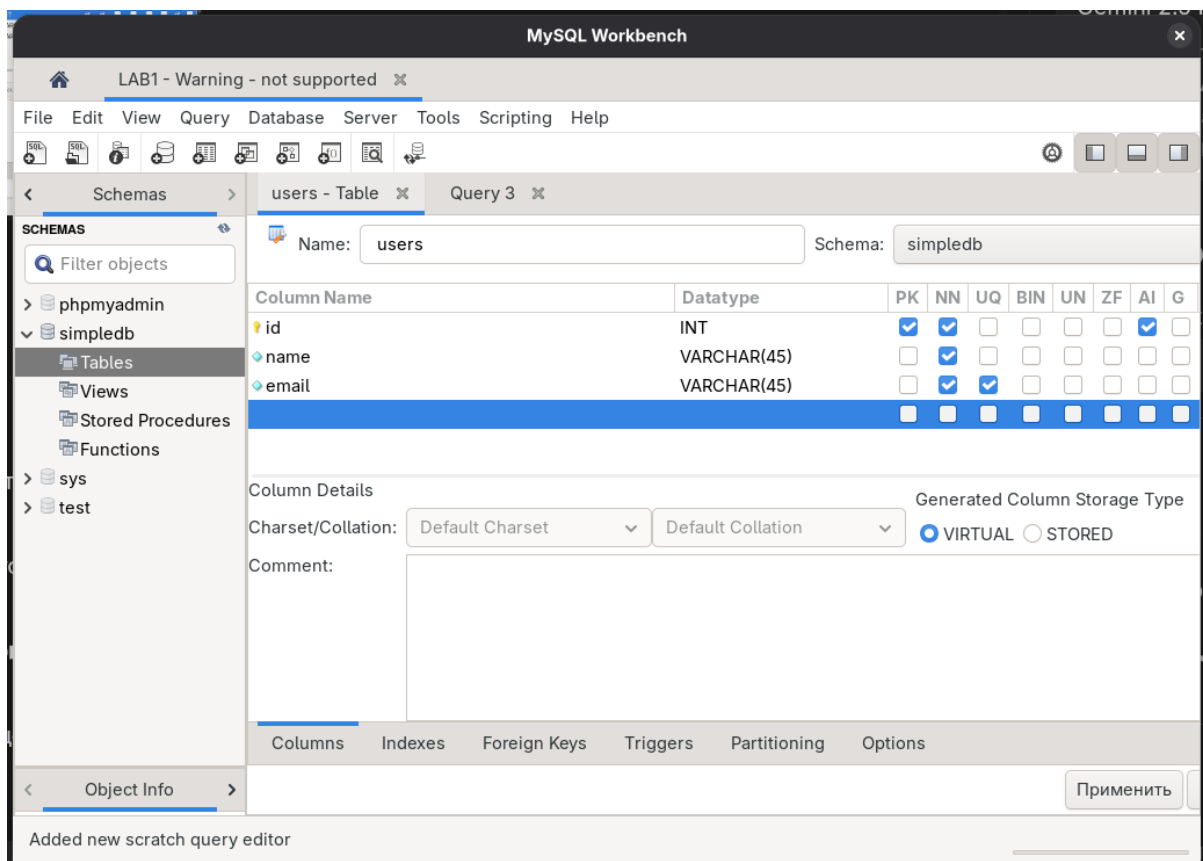
Задание 3:

```
CREATE TABLE users ( id int(11) NOT NULL AUTO_INCREMENT, name varchar(45) NOT NULL, email varchar(45) NOT NULL, PRIMARY KEY (id), UNIQUE KEY email_UNIQUE (email) ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3 COLLATE=utf8mb3_uca1400_ai_ci;
```

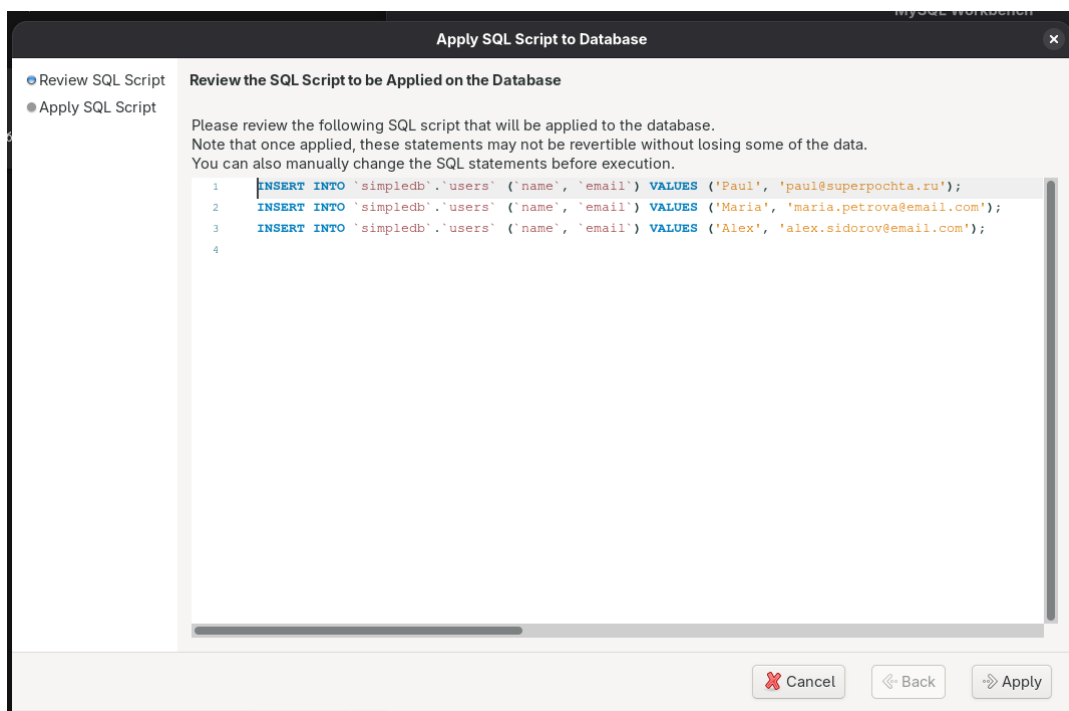
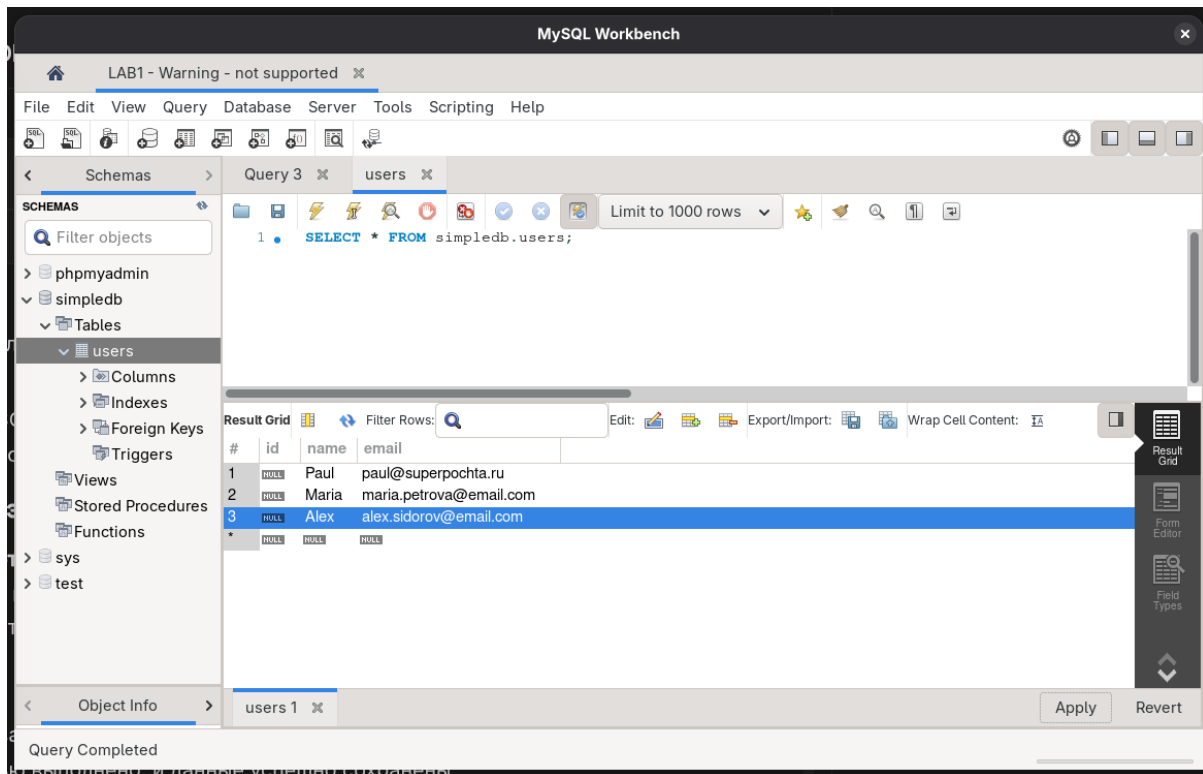


The screenshot shows the SQL editor in MySQL Workbench with the following SQL code:

```
1 CREATE TABLE `users` (  
2   `id` int(11) NOT NULL AUTO_INCREMENT,  
3   `name` varchar(45) NOT NULL,  
4   `email` varchar(45) NOT NULL,  
5   PRIMARY KEY (`id`),  
6   UNIQUE KEY `email_UNIQUE` (`email`)  
7 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3 COLLATE=utf8mb3_uca1400_ai_ci;  
8
```



Задание 4:



Задание 5:

Column Name	Datatype	PK	NN	UQ	BIN	UN	ZF	AI	G	Default / Expression
id	INT(11)	☒	☒	☐	☐	☐	☐	☒	☐	
name	VARCHAR(50)	☐	☒	☐	☐	☐	☐	☐	☐	
email	VARCHAR(45)	☐	☒	☒	☐	☐	☐	☐	☐	
gender	ENUM('M', 'F')	☐	☐	☐	☐	☐	☐	☐	☐	
bday	DATE	☐	☐	☐	☐	☐	☐	☐	☐	
postal_code	VARCHAR(10)	☐	☐	☐	☐	☐	☐	☐	☐	
rating	FLOAT	☐	☐	☐	☐	☐	☐	☐	☐	
created	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	CURRENT_TIMESTAMP()

SQL-запрос для изменения таблицы users:

```
ALTER TABLE `simpledb`.`users`
```

```
CHANGE COLUMN `name` `name` VARCHAR(50) NOT NULL ,
```

```
ADD COLUMN `gender` ENUM('M', 'F') NULL AFTER `email` ,
```

```
ADD COLUMN `bday` DATE NULL AFTER `gender` ,
```

```
ADD COLUMN `postal_code` VARCHAR(10) NULL AFTER `bday` ,
```

```
ADD COLUMN `rating` FLOAT NULL AFTER `postal_code` ,
```

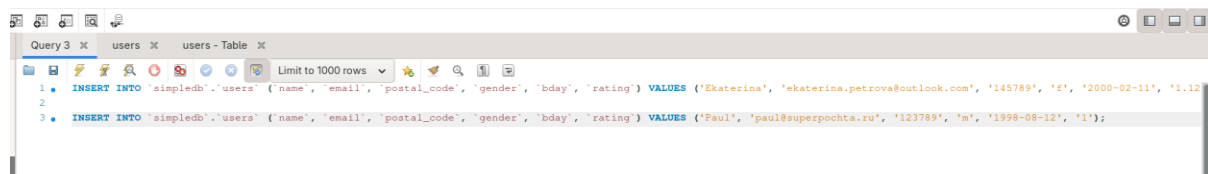
```
ADD COLUMN `created` TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP() AFTER  
`rating` ;
```

Описание и анализ (Замечания 1, 2, 3):

- **Тип ENUM:** Тип данных ENUM('M','F') для поля gender является перечислением и позволяет хранить только одно из двух указанных значений: 'M' или 'F'. Это обеспечивает целостность данных, так как в это поле невозможно записать никакое другое строковое значение.
- **Тип TIMESTAMP:** Тип TIMESTAMP для поля created хранит полную метку времени (дату и время). Установка значения по умолчанию CURRENT_TIMESTAMP() приводит к тому, что при создании новой записи, если не указать значение для этого поля, база данных автоматически подставит текущие системные дату и время. Это стандартная практика для отслеживания времени создания записей.
- **Поля со значением NULL:** Поля gender, bday, postal_code и rating могут иметь значение NULL (быть пустыми). Это сделано потому, что данная

информация является персональной, и пользователь может не захотеть её указывать при регистрации. Отсутствие ограничения NOT NULL для этих полей позволяет создавать записи пользователей без их обязательного заполнения.

Задание 6:



```
INSERT INTO `simplifiedb`.`users` (`name`, `email`, `postal_code`, `gender`,
`bday`, `rating`) VALUES ('Ekaterina', 'ekaterina.petrova@outlook.com', '145789', 'f',
'2000-02-11', '1.123');
```

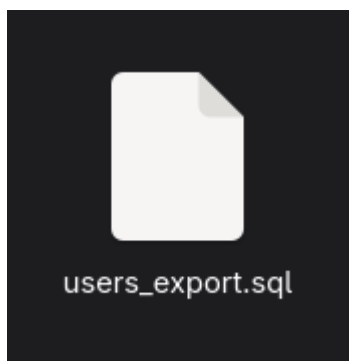
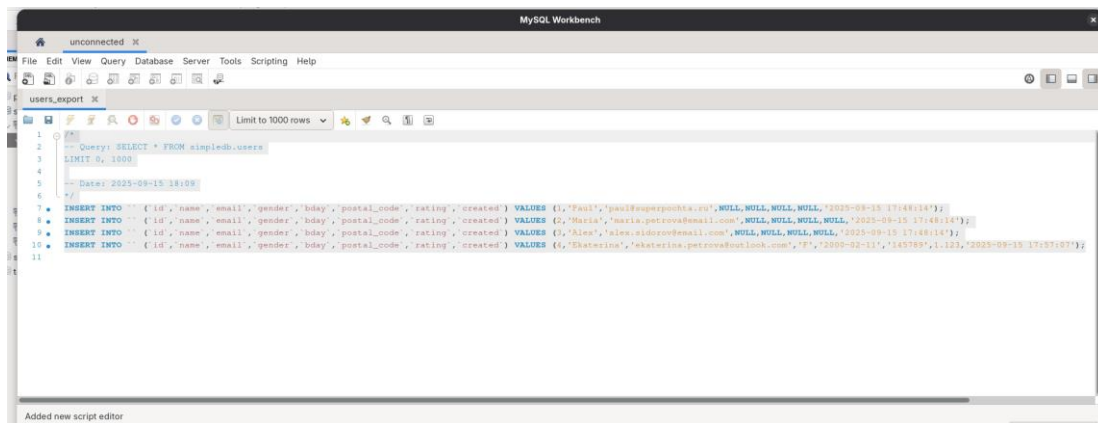
```
INSERT INTO `simplifiedb`.`users` (`name`, `email`, `postal_code`, `gender`,
`bday`, `rating`) VALUES ('Paul', 'paul@superpochta.ru', '123789', 'm', '1998-08-12',
'1');
```

#	id	name	email	gender	bday	postal_code	rating	created
1	1	Paul	paul@superpochta.ru	NULL	NULL	NULL	NULL	2025-09-15 17:48:14
2	2	Maria	maria.petrova@email.com	NULL	NULL	NULL	NULL	2025-09-15 17:48:14
3	3	Alex	alex.sidorov@email.com	NULL	NULL	NULL	NULL	2025-09-15 17:48:14
4	4	Ekaterina	ekaterina.petrova@outlook.com	F	2000-02-11	145789	1.123	2025-09-15 17:57:07
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

В ходе выполнения задания были исполнены два SQL-запроса INSERT для добавления в таблицу users двух новых пользователей.

1. Первый запрос, добавляющий пользователя 'Ekaterina' с уникальным email ekaterina.petrova@outlook.com, выполнен успешно. В таблицу была добавлена новая запись с id=4.
2. Второй запрос, который должен был добавить пользователя 'Paul' с email paul@superpochta.ru, **завершился ошибкой и не был выполнен**. Причиной этому послужило то, что в таблице уже существует запись (с id=1), имеющая такой же email. Сработало **ограничение уникальности (UNIQUE constraint)**, которое было установлено для столбца email на этапе проектирования таблицы.

Задание 7:



/*

-- Query: SELECT * FROM simplifiedb.users

LIMIT 0, 1000

-- Date: 2025-09-15 18:09

*/

INSERT INTO ` `

(`id`,`name`,`email`,`gender`,`bday`,`postal\_code`,`rating`,`created`) VALUES
(1,'Paul','paul@superpochta.ru',NULL,NULL,NULL,NULL,'2025-09-15 17:48:14');

INSERT INTO ` `

(`id`,`name`,`email`,`gender`,`bday`,`postal\_code`,`rating`,`created`) VALUES
(2,'Maria','maria.petrova@email.com',NULL,NULL,NULL,NULL,'2025-09-15 17:48:14');

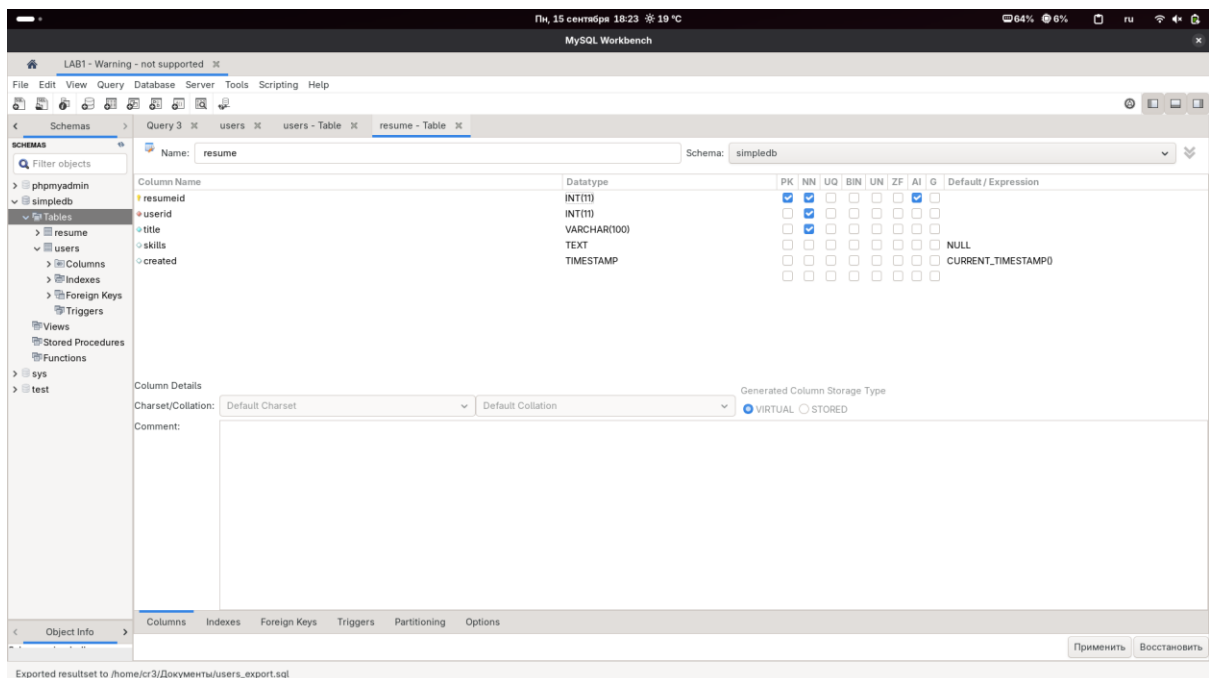
INSERT INTO ``

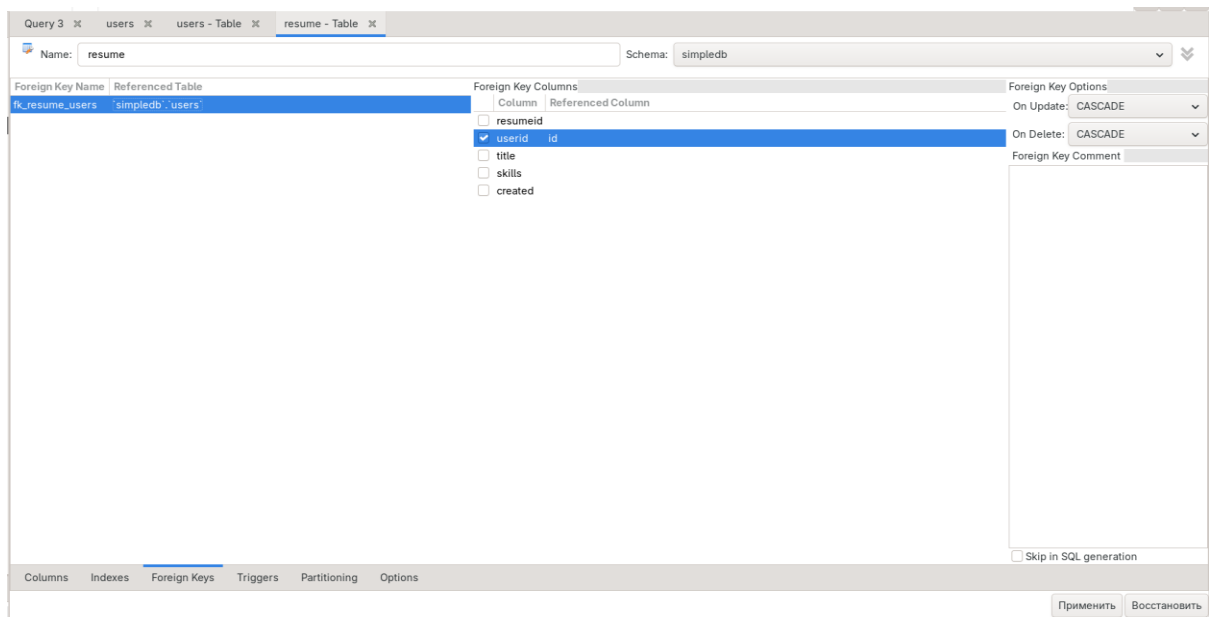
```
(`id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES  
(3,'Alex','alex.sidorov@email.com',NULL,NULL,NULL,NULL,'2025-09-15 17:48:14');
```

INSERT INTO ``

```
(`id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES  
(4,'Ekaterina','ekaterina.petrova@outlook.com','F','2000-02-11','145789',1.123,'2025-  
09-15 17:57:07');
```

Задание 8:





SQL-запрос для создания таблицы resume и внешнего ключа:

```
-- Создание таблицы resume CREATE TABLE simpledb.resume ( resumeid INT NOT NULL AUTO_INCREMENT, userid INT NOT NULL, title VARCHAR(100) NOT NULL, skills TEXT NULL, created TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP(), PRIMARY KEY (resumeid));
```

```
-- Добавление внешнего ключа для связи с таблицей users ALTER TABLE simpledb.resume ADD CONSTRAINT fk_resume_users FOREIGN KEY (userid) REFERENCES simpledb.users (id) ON DELETE CASCADE ON UPDATE CASCADE;
```

Данный код сначала создает таблицу `resume` со всеми необходимыми полями, а затем добавляет к ней внешний ключ, который связывает поле `userid` таблицы `resume` с полем `id` таблицы `users`.

Задание 9:

The screenshot shows a database management interface with a query result grid. The query is `SELECT * FROM simpledb.resume;`. The result grid displays the following data:

#	resumeid	userid	title	skills	created
1	1	1	Веб-разработчик	HTML, CSS, JavaScript	2025...
2	2	3	Аналитик данных	Python, Pandas, SQL	2025...
3	3	4	Менеджер проектов	Agile, Scrum, Jira	2025...

The screenshot shows the 'Apply SQL Script to Database' dialog box. The 'Apply SQL Script' tab is selected. The message log displays the following error:

Applying SQL script to the database

The following tasks will now be executed. Please monitor the execution.
Press Show Logs to see the execution logs.

☒ Execute SQL Statements

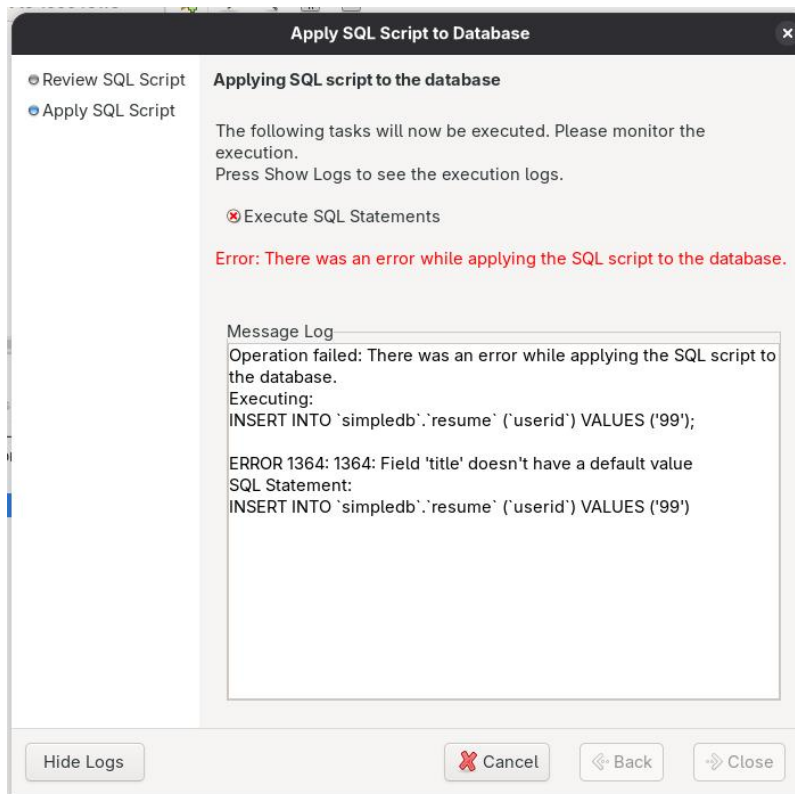
Error: There was an error while applying the SQL script to the database.

Message Log

Operation failed: There was an error while applying the SQL script to the database.
Executing:
`INSERT INTO `simpledb`.`resume` (`resumeid`, `userid`, `title`, `skills`) VALUES ('', '99', 'ыва', 'ука');`

ERROR 1366: 1366: Incorrect integer value: '' for column `simpledb`.`resume`.`resumeid` at row 1
SQL Statement:
`INSERT INTO `simpledb`.`resume` (`resumeid`, `userid`, `title`, `skills`) VALUES ('', '99', 'ыва', 'ука')`

Buttons: Hide Logs, Cancel, Back, Close



Анализ результатов и ответы на вопросы:

1. **Сколько резюме может быть у одного пользователя?** У одного пользователя может быть минимум 0 резюме, а максимум — неограниченное количество. Это является классическим примером связи "один ко многим" (one-to-many relationship).
2. **Попытка добавить резюме с несуществующим userid:** При попытке добавить запись в таблицу resume с userid, которого не существует в таблице users (например, 99), была получена ошибка (ERROR 1452: Cannot add or update a child row: a foreign key constraint fails). Это доказывает, что внешний ключ работает корректно. База данных не позволяет создать "дочернюю" запись (резюме), если для нее нет соответствующей "родительской" записи (пользователя). Это обеспечивает ссылочную целостность данных и предотвращает появление "осиротевших" записей в базе.

Задание 10:

```
DELETE FROM `simplifiedb`.`users` WHERE (`id` = '1');
```



В ходе выполнения Задания 10 из таблицы `users` был удален пользователь с `id=1`. После этого, при проверке таблицы `resume`, было обнаружено, что все записи, связанные с этим пользователем (где `userid=1`), также были автоматически удалены.

Это произошло благодаря правилу `ON DELETE CASCADE`, которое было установлено для внешнего ключа при создании таблицы `resume`. Данное правило обеспечивает целостность данных, автоматически удаляя связанные "дочерние" записи при удалении "родительской" записи.